



Rapport de TER (Travail d'Étude et de Recherche)

Master 1 Informatique - Parcours SeCRéTS

Sécurité des Contenus, des Réseaux, des Télécommunications et des Systèmes

Université Paris-Saclay - Campus UVSQ (Versailles)

Détection efficace de propriétés statistiques de chiffrements symétriques

Présenté par :

Alexandre FRANÇOIS
Binet DUPRÉ
Bastien GUIBERT

Sous la direction de :

Jules BAUDRIN
Laboratoire de Mathématiques de Versailles (LMV) - UVSQ

Université Paris-Saclay
Année universitaire 2025-2026
4 avril 2026

Table des matières

0.1	L'Algorithme Naïf Absolu (Recherche Exhaustive)	2
0.1.1	Principe Théorique (Version Brute)	2
0.1.2	Complexités de l'Algorithme Brut	2
0.1.3	De la théorie à la pratique : Nos optimisations	2
0.1.4	Pseudo-code de notre implémentation	3
0.1.5	Limites matérielles et Stratégies d'adaptation	3
0.2	L'Algorithme de Recherche Standard : Une approche statistique	4
0.2.1	Principes et Fondements Statistiques : Distinguer le signal du bruit	4
0.2.2	Analyse de l'implémentation C++ et Limites	5
0.3	L'Algorithme Fondamental : L'exploitation des collisions	5
0.3.1	Le concept de Différenciation par Substitut	5
0.3.2	Le mécanisme du « Quatuor Correct » (<i>Right Quartet</i>)	5
0.3.3	Gain de complexité et dimensionnement de l'échantillon M	6
0.3.4	Démonstration mathématique des seuils de détection et vérification	6
0.3.5	Implémentation, Complexités et Limite de la Mémoire Vive	8
0.4	L'Algorithme Sans Mémoire (<i>Memoryless</i>) : Le prix du temps	9
0.5	L'Algorithme de Compromis Temps/Mémoire (Tradeoff)	9
0.5.1	Principe Théorique : Échantillonnage par lots (<i>Batching</i>)	10
0.5.2	De la théorie à la pratique : Notre implémentation C++	11
0.5.3	Ouverture : La dépendance aux hypothèses	11
0.6	L'Algorithme « Pire Cas » (<i>Worst-Case</i>) : S'affranchir des heuristiques	12
0.6.1	Paramétrage et seuils de validation	12
0.6.2	Complexités théoriques et Implémentation	13
0.6.3	Limites pratiques : Le cumul des contraintes	13
0.7	Choix d'implémentation : Optimisation pour les blocs de 32 bits	14
0.8	Validation expérimentale et résolution matérielle	15

0.1 L'Algorithme Naïf Absolu (Recherche Exhaustive)

La méthode la plus directe pour détecter des propriétés statistiques au sein d'un chiffrement symétrique consiste à évaluer de manière exhaustive toutes les paires possibles de textes clairs. Cet algorithme, que nous qualifierons de « naïf absolu », sert de point de référence théorique pour comprendre la mécanique de la cryptanalyse différentielle et construire la **Table de Distribution des Différences (DDT)**.

0.1.1 Principe Théorique (Version Brute)

Dans sa version conceptuelle la plus pure, l'algorithme ne possède aucun filtre ni optimisation. Pour chaque différence d'entrée possible α (allant de 1 à $2^n - 1$, où n est la taille du bloc en bits), il parcourt l'intégralité de l'espace des messages x (soit 2^n messages). La boucle sur α commence délibérément à 1 et non à 0, car une différence $\alpha = 0$ implique des messages identiques ($x \oplus 0 = x$), conduisant toujours à une différence de sortie triviale $\beta = 0$, sans aucun intérêt cryptanalytique.

Pour chaque message x , la version brute calcule les chiffrés à la volée, évalue la différence de sortie $\beta = F(x) \oplus F(x \oplus \alpha)$, et incrémente la case correspondante dans une matrice globale (la DDT complète de taille $2^n \times 2^n$).

0.1.2 Complexités de l'Algorithme Brut

L'évaluation de cette approche naïve met en évidence des limites drastiques :

- **Complexité temporelle** : $\mathcal{O}(2^{2n})$. Il y a approximativement 2^n différences d'entrées α à tester, et pour chacune d'elles on évalue 2^n messages x , ce qui nécessite 2×2^{2n} opérations de chiffrement. Cette complexité quadratique rend la méthode strictement inutilisable en temps raisonnable dès que $n > 16$.
- **Complexité en données** : $\mathcal{O}(2^n)$. L'algorithme nécessite le chiffrement de l'ensemble de l'espace des textes clairs possibles.
- **Complexité spatiale (mémoire)** : $\mathcal{O}(2^{2n})$. Stocker la DDT complète en mémoire requiert une matrice de taille $2^n \times 2^n$ entiers, ce qui sature instantanément la mémoire vive pour des tailles de blocs standards.

0.1.3 De la théorie à la pratique : Nos optimisations

Pour pouvoir exécuter cette recherche exhaustive efficacement sur des « chiffrements jouets » ($n \leq 16$), l'implémentation théorique brute a dû être repensée. L'intégralité de notre code optimisé, développé en C++, est disponible sur notre dépôt GitHub public (<https://github.com/Redak92/cryptanalyse-differentielle>), dans le fichier `src/analysis/DifferentialSearch.cpp`.

Nous avons introduit trois optimisations majeures pour rendre l'algorithme praticable :

1. **Pré-calcul des chiffrés** : Pour réduire la surcharge temporelle liée aux appels répétés à la fonction de chiffrement, nous avons mis en place un compromis temps/mémoire (*time/memory trade-off*). L'ensemble des 2^n messages possibles est évalué une seule fois au début de l'exécution, et les résultats sont stockés dans une table de correspondance globale (ou *lookup table*), que nous noterons T . Le calcul de la différence de sortie β se résume alors à deux simples accès mémoire directs suivis d'une disjonction exclusive : $T[x] \oplus T[x \oplus \alpha]$.
2. **Filtrage à la volée (Génération de pDDT)** : En cryptanalyse différentielle, nous ne nous intéressons qu'aux différentielles à haute probabilité, qui traduisent une véritable faiblesse de la primitive. Si le chiffrement est un minimum bien conçu, la vaste majorité des entrées d'une DDT classique n'est constituée que de bruit statistique sans valeur exploitable. Notre algorithme s'aligne sur ce principe en prenant en paramètre un seuil de probabilité pour ne générer qu'une **DDT partielle (pDDT)**. L'algorithme utilise un tableau

temporaire de compteurs de taille 2^n et, à la fin de chaque itération sur α , ne sauvegarde que les différentielles atteignant la probabilité requise. En éliminant tout ce bruit inutile, l’empreinte mémoire de la structure finale est drastiquement réduite, évitant ainsi le stockage d’une matrice en $\mathcal{O}(2^{2n})$.

3. **Parallélisation (OpenMP)** : L’itération principale parcourant les différences α fait l’objet d’une parallélisation. Chaque thread CPU alloue son propre sous-tableau de compteurs pour éviter les accès concurrents, ce qui divise drastiquement le temps d’exécution sur les architectures multi-cœurs.

0.1.4 Pseudo-code de notre implémentation

Le pseudo-code suivant résume l’algorithme tel qu’il a été implémenté et optimisé dans notre code source.

Algorithm 1 Recherche Différentielle Naïve Optimisée (Génération de pDDT)

Entrées : F (fonction de chiffrement), n (taille du bloc), p (seuil de probabilité)

Sortie : Liste des différentielles (α, β) ayant une probabilité $\geq p$

```

1:  $limit \leftarrow 2^n$ 
2:  $minCount \leftarrow p \times limit$ 
3: Initialiser un tableau  $f_{table}$  de taille  $limit$ 
4: // Remplissage de la  $f_{table}$  (Pré-calculs)
5: for  $x \leftarrow 0$  to  $limit - 1$  do
6:    $f_{table}[x] \leftarrow F(x)$ 
7: end for
8: // Boucle parallélisée (OpenMP)
9: for  $\alpha \leftarrow 1$  to  $limit - 1$  do
10:  Initialiser un tableau  $counts$  de taille  $limit$  à 0
11:  for  $x \leftarrow 0$  to  $limit - 1$  do
12:     $\beta \leftarrow f_{table}[x] \oplus f_{table}[x \oplus \alpha]$ 
13:     $counts[\beta] \leftarrow counts[\beta] + 1$ 
14:  end for
15:  for  $\beta \leftarrow 0$  to  $limit - 1$  do
16:    if  $counts[\beta] > minCount$  then
17:      Sauvegarder  $(\alpha, \beta, \frac{counts[\beta]}{limit})$ 
18:    end if
19:  end for
20: end for
```

0.1.5 Limites matérielles et Stratégies d’adaptation

Bien que très efficace sur des chiffrements jouets, cette implémentation se heurte rapidement au « mur de la RAM » lorsque n augmente. Par exemple, pour analyser Speck 32/64 ($n = 32$), le simple fait de stocker la f_{table} (contenant 2^{32} blocs de 64 bits, soit 8 octets par bloc) nécessite $2^{32} \times 8 \approx 34,36$ Go de RAM, provoquant la saturation immédiate d’un ordinateur personnel standard.

Lorsque la taille du bloc menace les capacités matérielles, l’algorithme doit être adapté en sacrifiant la vitesse d’exécution pour préserver la mémoire :

- **Suppression du pré-calcul (Évaluation à la volée)** : C’est la solution la plus radicale pour sauver la RAM. En supprimant totalement l’ f_{table} , l’algorithme ne stocke plus rien au préalable. Il calcule $F(x)$ et $F(x \oplus \alpha)$ à chaque itération. L’empreinte spatiale redevient minime, mais le temps d’exécution explose (la complexité temporelle repassant à son maximum quadratique).

- **Désactiver la parallélisation** : Une option d'allègement consiste à repasser en mode séquentiel. On évite ainsi l'allocation multiple des grands tableaux de compteurs par les différents threads CPU.
- **Nettoyage à la volée (*flush & clear*)** : Au lieu de stocker toutes les différentielles valides en mémoire dans une unique structure de résultats finale, l'algorithme affiche immédiatement sur la sortie standard les paires trouvées pour un α donné, puis vide et réinitialise sa mémoire avant de passer au α suivant.

0.2 L'Algorithme de Recherche Standard : Une approche statistique

Bien que l'algorithme exhaustif soit utile pour des paramètres très réduits, sa complexité en $\mathcal{O}(2^{2n})$ devient extrêmement lourde pour des chiffrements traitant des blocs de 32 bits, et totalement impraticable au-delà. Pour pallier cette limitation, une approche naturelle consiste à transformer la recherche déterministe en un problème d'échantillonnage statistique. Cet algorithme est décrit par Dinur et al. comme l'adaptation classique de la construction d'une DDT pour détecter des différentielles de probabilité supérieure à un seuil p .

0.2.1 Principes et Fondements Statistiques : Distinguer le signal du bruit

Plutôt que de sonder l'immensité de l'espace des messages, l'algorithme privilégie une stratégie par échantillonnage. Pour chaque différence d'entrée α , nous testons un nombre fixe k de paires aléatoires. Cette approche transforme la recherche en un défi statistique où l'objectif est de faire émerger un « signal » (une différentielle réelle) d'un « bruit » de fond aléatoire.

Le nombre de succès X , représentant le nombre de paires dont la différence de sortie est effectivement β , suit initialement une loi binomiale $\mathcal{B}(k, p)$. Cependant, face à un nombre de tirages k élevé et une probabilité de succès p très faible, nous basculons dans le domaine de la **loi des événements rares**. La distribution de X peut alors être approximée par une **loi de Poisson** $\mathcal{P}(\lambda)$. Dans ce modèle, le paramètre λ représente l'espérance mathématique du nombre de succès, soit $\lambda = k \times p$. La probabilité d'obtenir exactement x succès est donnée par la formule $P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$.

Pour garantir que nous ne passerons pas à côté d'une différentielle significative, les auteurs fixent $k = 4/p$ ($\lambda = 4$). La probabilité de détection se calcule par le complémentaire :

$$P(X \geq 2) = 1 - (P(X = 0) + P(X = 1)) = 1 - \left(\frac{\lambda^0 e^{-\lambda}}{0!} + \frac{\lambda^1 e^{-\lambda}}{1!} \right) = 1 - e^{-\lambda}(1 + \lambda)$$

Pour $\lambda = 4$, on obtient $1 - 5e^{-4} \approx 0,9085$, validant le seuil de confiance de 90 % de la littérature.

Le choix du critère $\text{count} \geq 2$ est crucial pour séparer le signal du bruit. Pour une permutation aléatoire, la probabilité d'une collision fortuite est extrêmement faible ($p_{\text{rand}} = \frac{1}{2^n - 1}$). Avec Speck 64 ($n = 64$) et $p = 2^{-13}$, l'espérance du bruit $\lambda_{\text{rand}} = k \times p_{\text{rand}}$ est d'environ 2^{-49} .

Pour évaluer la probabilité qu'un tel bruit génère un faux positif, nous utilisons le **développement limité de Taylor** de l'exponentielle au voisinage de zéro : $e^{-\lambda} \approx 1 - \lambda + \frac{\lambda^2}{2}$. La démonstration complète de ce développement est disponible sur le site *Math-Linux*¹.

L'usage de ce développement est ici indispensable car λ est extrêmement petit, il permet de simplifier l'exponentielle en une simple puissance, ce qui permet d'évaluer facilement des probabilités extrêmement faibles. En injectant cette approximation dans notre formule factorisée, le calcul devient :

$$P(X \geq 2) \approx 1 - \left(1 - \lambda + \frac{\lambda^2}{2} \right) (1 + \lambda) = 1 - \left(1 + \lambda - \lambda - \lambda^2 + \frac{\lambda^2}{2} + \frac{\lambda^3}{2} \right)$$

1. Lien vers la démonstration du DL de l'exponentielle

En négligeant le terme en λ^3 (d'ordre supérieur et donc totalement insignifiant face à λ^2 quand $\lambda \rightarrow 0$), on aboutit à :

$$P(X \geq 2) \approx 1 - \left(1 - \frac{\lambda^2}{2}\right) = \frac{\lambda^2}{2}$$

Dans notre exemple, cette probabilité de faux signal chute à environ 2^{-99} , ce qui rend toute collision accidentelle statistiquement impossible et prouve l'existence d'une structure non aléatoire.

0.2.2 Analyse de l'implémentation C++ et Limites

Notre implémentation, disponible dans le fichier `src/analysis/DifferentialSearch.cpp`, parallélise l'itération sur les différences d'entrée α en répartissant la charge sur plusieurs fils d'exécution. Contrairement à l'algorithme exhaustif, l'utilisation d'une table de hachage dynamique permet de ne conserver que les différences de sortie effectivement rencontrées, optimisant ainsi considérablement l'empreinte mémoire.

Toutefois, ce moteur reste limité par le facteur 2^n . Pour un chiffrement comme **Speck 64/128**, l'algorithme doit itérer, à minima, 2^{64} fois, ce qui est prohibitif sur un ordinateur personnel.

C'est cette barrière qui justifie l'introduction de l'algorithme probabiliste présenté dans l'article. En s'appuyant sur le **paradoxe des anniversaires**, ce dernier permet de convertir la recherche de différentielles en une recherche de collisions, abaissant la complexité à l'ordre de $2^{n/2} \cdot p^{-1}$.

0.3 L'Algorithme Fondamental : L'exploitation des collisions

Pour franchir la barrière du 2^n , cet algorithme délaisse l'approche par différence d'entrée fixe pour s'appuyer sur la **différenciation par substitut**. L'objectif est de détecter toutes les différentielles $\alpha \rightarrow \beta$ de probabilité p avec une complexité temporelle et spatiale réduite à l'ordre de $\tilde{O}(2^{n/2} \cdot p^{-1})$.

0.3.1 Le concept de Différenciation par Substitut

L'idée novatrice du papier consiste à choisir une différence γ arbitraire et non nulle, appelée **substitut**, totalement indépendante de la différence α recherchée. On définit la fonction g_γ comme la dérivée de f dans la direction du substitut :

$$g_\gamma(x) = f(x) \oplus f(x \oplus \gamma)$$

Pour comprendre intuitivement ce mécanisme, on peut comparer la recherche de différentielles à la recherche d'aiguilles dans une botte de foin. L'approche standard reviendrait à scanner le foin centimètre par centimètre en changeant l'angle du détecteur pour chaque direction α possible, ce qui représente un travail linéaire épuisant de l'ordre de 2^n . À l'inverse, la différenciation par substitut traite tous les angles α simultanément en agissant comme un filtre global appliqué à toute la botte. Ce filtre ne trouve pas directement l'aiguille (β), mais il crée une « empreinte » (g_γ) pour chaque brin de foin. Si deux brins partagent la même empreinte, cela révèle mathématiquement l'existence d'une structure alignée, soit la direction α , sans que l'on ait eu besoin de tester chaque angle individuellement.

0.3.2 Le mécanisme du « Quatuor Correct » (*Right Quartet*)

Le succès de l'algorithme repose sur une structure géométrique reliant quatre messages, formant ce que les auteurs appellent un quatuor $\{x, x \oplus \alpha, x \oplus \gamma, x \oplus \alpha \oplus \gamma\}$. Un tel quatuor est dit **correct** pour la différentielle $\alpha \rightarrow \beta$ si le couple initial $(x, x \oplus \alpha)$ et le couple translaté par le substitut $(x \oplus \gamma, x \oplus \gamma \oplus \alpha)$ satisfont simultanément cette différentielle. Autrement dit, on doit observer à la fois $f(x) \oplus f(x \oplus \alpha) = \beta$ et $f(x \oplus \gamma) \oplus f(x \oplus \gamma \oplus \alpha) = \beta$.

La fonction g_γ permet alors d'annuler la valeur de β par le biais d'une double différenciation. En calculant la différence entre deux sorties de g_γ espacées de α , nous obtenons :

$$\begin{aligned} g_\gamma(x) \oplus g_\gamma(x \oplus \alpha) &= [f(x) \oplus f(x \oplus \gamma)] \oplus [f(x \oplus \alpha) \oplus f(x \oplus \alpha \oplus \gamma)] \\ &= [f(x) \oplus f(x \oplus \alpha)] \oplus [f(x \oplus \gamma) \oplus f(x \oplus \alpha \oplus \gamma)] \\ &= \beta \oplus \beta = 0 \end{aligned}$$

Cette égalité fondamentale implique que $g_\gamma(x) = g_\gamma(x \oplus \alpha)$, créant ainsi une **collision** dans la table de hachage de l'algorithme. Le point crucial de cette méthode est que la collision se produit **quelle que soit la valeur de β** . L'algorithme détecte donc la différence d'entrée α simplement en cherchant des doublons dans les sorties de g_γ , et ne calcule la différence de sortie β qu'a posteriori via l'opération $\beta = f(x) \oplus f(x \oplus \alpha)$.

0.3.3 Gain de complexité et dimensionnement de l'échantillon M

Maintenant que nous savons qu'une différentielle se manifeste par une collision dans g_γ , la question centrale de l'algorithme devient : combien de messages aléatoires, que nous noterons M , devons-nous générer et évaluer pour observer cette collision avec certitude ?

C'est ici qu'intervient le **paradoxe des anniversaires**. Dans l'approche standard, un message a une probabilité p de satisfaire la différentielle. Dans cette nouvelle approche, puisqu'un quatuor correct exige que deux couples distincts satisfassent simultanément la propriété, un message aléatoire x a une probabilité p^2 d'appartenir à un tel quatuor.

Pour garantir la détection d'une différentielle réelle, il est donc nécessaire de collecter un échantillon de taille M suffisamment large pour qu'un nombre significatif de quatuors corrects (et donc de collisions) apparaisse. Plus précisément, les auteurs fixent l'objectif d'obtenir une espérance de $E = n/2$ collisions pour une différentielle de probabilité p .

La dérivation de cette quantité M découle directement des principes probabilistes fondamentaux. Avec un ensemble de M messages, il est possible de former approximativement $\frac{M^2}{2}$ paires distinctes (x_i, x_j) . Pour une paire donnée, la probabilité que la différence $x_i \oplus x_j$ corresponde exactement à l'unique valeur α recherchée est de 2^{-n} . De plus, sous l'hypothèse d'indépendance, la probabilité que les deux couples formés par ce quatuor satisfassent simultanément la différentielle est égale à p^2 . Par conséquent, l'espérance du nombre de collisions suggérant la bonne différentielle s'exprime comme le produit de ces facteurs :

$$E[\text{Collisions}] = \frac{M^2}{2} \cdot \frac{1}{2^n} \cdot p^2$$

Afin d'assurer un signal statistiquement robuste, cette espérance est fixée à $n/2$. En résolvant cette équation pour M , on obtient le dimensionnement optimal :

$$\frac{M^2 \cdot p^2}{2 \cdot 2^n} = \frac{n}{2} \implies M^2 = \frac{n \cdot 2^n}{p^2} \implies M = \sqrt{n} \cdot 2^{n/2} \cdot p^{-1}$$

Ce résultat démontre mathématiquement comment le passage d'une recherche exhaustive à une recherche de collisions globales permet de réduire la complexité temporelle globale d'un facteur $2^{n/2}$.

0.3.4 Démonstration mathématique des seuils de détection et vérification

Une fois l'échantillon M généré, l'algorithme s'appuie sur trois seuils statistiques précis pour séparer le signal du bruit : $n/4$ pour la détection, puis n/p et $n/2$ pour la vérification. Ces valeurs ne sont pas empiriques, elles découlent d'une analyse rigoureuse basée sur la loi de Poisson.

Phase de détection : Le seuil $n/4$ Pour une vraie différentielle de probabilité p , l'espérance du nombre de collisions (quatuors corrects) parmi les M messages évalués est $\lambda = n/2$. Fondamentalement, la détection de chaque quatuor correspond à une épreuve de Bernoulli indépendante, ce qui signifie que le nombre total de collisions observées, que nous noterons X , suit rigoureusement une **loi binomiale**. Cependant, face au nombre gigantesque de tirages (l'espace des messages) et à la rareté d'un succès (la probabilité p^2 étant infime), cette distribution binomiale s'approxime parfaitement par une **loi de Poisson** de paramètre $\lambda = n/2$. On note ainsi :

$$X \sim \mathcal{P}\left(\frac{n}{2}\right)$$

L'algorithme configure son filtre pour ne conserver que les candidats (α, β) ayant généré au moins $n/4$ collisions. Pour prouver la robustesse de ce choix, il faut évaluer son comportement face au signal légitime, c'est-à-dire la validation des vrais positifs. La probabilité qu'une vraie différentielle soit correctement détectée correspond à $P(X \geq n/4)$. Puisque le seuil d'acceptation ($n/4$) est fixé exactement à la moitié de l'espérance théorique ($n/2$), la probabilité de rejeter à tort un candidat valide correspond à $P(X < n/4)$. Mathématiquement, cette probabilité d'échec s'obtient par le calcul direct de la fonction de répartition de la loi de Poisson :

$$P\left(X < \frac{n}{4}\right) = \sum_{k=0}^{\frac{n}{4}-1} e^{-\frac{n}{2}} \frac{\left(\frac{n}{2}\right)^k}{k!}$$

L'évaluation exacte de cette somme finie montre que l'erreur s'effondre très rapidement à mesure que la taille du bloc n augmente. Par exemple, en appliquant ce calcul pour $n = 64$ (soit une espérance $\lambda = 32$ et un seuil d'acceptation de 16), l'erreur devient négligeable et le taux de réussite (vrais positifs) atteint 99,9 %.

À l'inverse, l'algorithme doit être capable de rejeter le bruit statistique (les faux positifs). Considérons une fausse différentielle, définie dans la littérature comme une propriété dont la probabilité est au plus $p/10$. Étant donné que la probabilité d'obtenir un quatuor correct évolue de façon quadratique par rapport à la probabilité de la différentielle, l'espérance des collisions pour cette fausse piste s'effondre drastiquement :

$$\lambda_{false} = \frac{n}{2} \cdot \left(\frac{p/10}{p}\right)^2 = \frac{n}{2} \cdot \frac{1}{100} = \frac{n}{200}$$

La probabilité qu'une telle anomalie statistique franchisse le filtre de détection est $P(Y \geq n/4)$, où $Y \sim \mathcal{P}(n/200)$. Atteindre le seuil de $n/4$ signifierait observer 50 fois l'espérance théorique ($n/200$). En évaluant la probabilité cumulée sur cette loi de Poisson pour un écart aussi extrême vers le haut, on démontre que le risque est strictement inférieur à 2^{-n} . Ainsi, la probabilité qu'une fausse piste soit validée par hasard est si infime qu'elle devient virtuellement impossible, garantissant la pureté des candidats transmis à la phase de vérification.

Phase de vérification : Les seuils n/p et $n/2$ Bien que le filtre de détection élimine l'immense majorité du bruit, l'algorithme sécurise définitivement le résultat en testant les candidats survivants. Pour ce faire, il génère un nouvel échantillon de test, totalement indépendant du premier, composé de $N_{verify} = n/p$ paires aléatoires présentant la différence d'entrée α suspectée.

Si le candidat est bel et bien une vraie différentielle de probabilité p , chaque paire testée constitue une nouvelle épreuve de Bernoulli avec une probabilité de succès p . Le nombre de succès Z (c'est-à-dire les cas où la différence de sortie est effectivement β) suit alors une loi binomiale qui, tout comme dans la phase de détection, s'approxime par une loi de Poisson. L'espérance de cette distribution est :

$$E[Z] = \lambda_{verify} = N_{verify} \times p = \left(\frac{n}{p}\right) \times p = n$$

L'algorithme valide définitivement la différentielle si le compteur de succès dépasse strictement le seuil de $n/2$. En plaçant à nouveau ce seuil d'acceptation à la moitié de l'espérance théorique, la logique mathématique reste identique à celle de la détection : la probabilité qu'une vraie différentielle échoue à ce stade (c'est-à-dire $P(Z \leq n/2)$) est négligeable et s'effondre avec la croissance de n .

À l'inverse, l'algorithme doit bloquer les fausses différentielles (de probabilité $p/10$ ou moins) qui auraient pu franchir la première étape par pure malchance statistique. Pour un tel candidat, l'espérance lors de cette phase de vérification s'écroule :

$$\lambda_{false_verify} = N_{verify} \times \frac{p}{10} = \left(\frac{n}{p}\right) \times \frac{p}{10} = \frac{n}{10}$$

Pour qu'un tel faux positif soit validé, il faudrait qu'il produise plus de $n/2$ succès, soit observer au moins 5 fois son espérance théorique. Les propriétés de la loi de Poisson confirment que la probabilité d'une telle déviation est tout aussi négligeable que dans la phase de détection. Cette double barrière statistique garantit la fiabilité absolue des propriétés remontées par l'algorithme.

0.3.5 Implémentation, Complexités et Limite de la Mémoire Vive

Le pseudo-code détaillé de cet algorithme fondamental est fourni à la page 9 de l'article. Nous l'avons intégralement implémenté dans notre programme sous la fonction `runFundamentalAlgorithm`, dont le code source C++ est librement accessible sur notre dépôt GitHub public, au sein du fichier `src/analysis/DifferentialSearch.cpp`.

Grâce à cette approche exploitant la différenciation par substitut, la complexité temporelle de la recherche est drastiquement abaissée à $\mathcal{O}(n \cdot 2^{n/2} \cdot p^{-1})$. Toutefois, cette version se heurte à une contrainte matérielle majeure : **l'empreinte mémoire**. Le maintien en mémoire d'une table de hachage globale pour stocker les M évaluations devient extrêmement prohibitif à mesure que la taille du bloc n augmente ou que le seuil de probabilité p diminue.

Pour illustrer théoriquement ce « mur de la RAM », calculons l'espace mémoire brut requis pour analyser un chiffrement aux paramètres pourtant modestes, tel que Speck 32/64 ($n = 32$), avec un seuil de probabilité $p = 2^{-13}$. Le nombre de messages M à évaluer et à stocker en phase de détection s'élève à :

$$M = \sqrt{n} \cdot 2^{n/2} \cdot p^{-1} = \sqrt{32} \cdot 2^{16} \cdot (2^{-13})^{-1} = \sqrt{32} \cdot 2^{29} \approx 3 \times 10^9 \text{ messages}$$

Sur le plan purement théorique, stocker chaque évaluation nécessite a minima de conserver le message d'entrée x et son résultat $g_\gamma(x)$ pour pouvoir identifier les collisions et en déduire α . Pour un bloc de 32 bits, cela représente 64 bits de données (soit 8 octets) par message. L'allocation de mémoire théorique brute, sans même compter le poids de la structure de données sous-jacente, s'élève donc à :

$$\text{Mémoire théorique} \approx 3 \times 10^9 \text{ messages} \times 8 \text{ octets/message} \approx 24 \times 10^9 \text{ octets} \approx \mathbf{24 \text{ Go}}$$

Ainsi, même pour analyser un chiffrement « jouet » de 32 bits, l'algorithme exige théoriquement un minimum absolu de 24 Go de mémoire vive. À titre de comparaison, passer à un bloc standard de 64 bits avec la même probabilité ferait exploser ce besoin brut à plusieurs centaines de Téraoctets.

Cette limitation physique absolue justifie la nécessité de recourir à des variantes de l'algorithme plus économes en ressources. Pour contourner ce « mur de la RAM », les auteurs explorent deux voies successives : une première approche radicale supprimant totalement le stockage, suivie d'une approche plus équilibrée reposant sur un compromis temps/mémoire.

0.4 L'Algorithme Sans Mémoire (*Memoryless*) : Le prix du temps

Pour pallier le problème rédhibitoire de l'empreinte spatiale de l'algorithme fondamental, les auteurs présentent à la page 13 de leur article une variante dite « sans mémoire » (*memoryless*). Cette approche permet de ramener la consommation de mémoire vive à une constante $\mathcal{O}(1)$, éliminant de facto le « mur de la RAM ».

Pour réussir ce tour de force, l'algorithme abandonne complètement la génération d'un large échantillon stocké dans une table de hachage. À la place, il utilise l'algorithme de détection de cycles de Pollard pour trouver des collisions dans la fonction g_γ à la volée, une par une. Chaque fois qu'une collision est détectée, la différence (α, β) correspondante est immédiatement extraite et soumise à la phase de vérification exhaustive. L'algorithme ne stocke aucun historique.

Cependant, cette économie spatiale drastique se paie au prix fort sur le plan des performances. Puisque l'algorithme perd la vision globale offerte par le paradoxe des anniversaires sur un grand tableau, il doit tester aveuglément de très nombreuses collisions inutiles avant de tomber sur un quatuor correct. Plus précisément, il faut extraire en moyenne $\mathcal{O}(p^{-2})$ collisions de la fonction g_γ pour obtenir une correspondance valide. La recherche d'une collision par la méthode de Pollard coûtant $\mathcal{O}(2^{n/2})$, la complexité temporelle globale grimpe à :

$$\text{Temps}_{\text{memoryless}} = \mathcal{O}(2^{n/2}) \times \mathcal{O}(p^{-2}) = \mathcal{O}(2^{n/2}p^{-2})$$

(ceci étant valable dans le cas où $p \geq 2^{-n/2}$).

Si l'on compare cette complexité à celle de l'algorithme fondamental, le rapport des temps d'exécution met en évidence l'ampleur du ralentissement :

$$\frac{\text{Temps}_{\text{memoryless}}}{\text{Temps}_{\text{fondamental}}} = \frac{2^{n/2}p^{-2}}{2^{n/2}p^{-1}} = p^{-1}$$

L'approche sans mémoire subit donc un **ralentissement direct d'un facteur p^{-1}** .

Pour bien mesurer l'impact de ce facteur, prenons un exemple concret sur le chiffrement Speck 32/64 ($n = 32$) avec une différentielle de probabilité $p = 2^{-13}$. Le calcul de la pénalité temporelle donne :

$$p^{-1} = (2^{-13})^{-1} = 2^{13} = 8192$$

Concrètement, cela signifie qu'une recherche qui prendrait **1 minute** avec l'algorithme fondamental prendrait 8192 fois plus de temps avec cette variante sans mémoire. En détaillant la conversion :

$$8192 \text{ minutes} = \frac{8192}{60} \text{ heures} \approx 136,5 \text{ heures} \approx \mathbf{5,68 \text{ jours}}$$

Et plus la probabilité p recherchée est faible, plus ce gouffre temporel se creuse. Bien que hautement parallélisable, cette méthode s'avère donc beaucoup trop lente pour être exploitée efficacement sur des machines standards, qui disposent pourtant de plusieurs gigaoctets de RAM laissés inutilisés par cette approche. C'est précisément pour combler ce vide entre le « tout mémoire » (incalculable) et le « sans mémoire » (interminable) que la solution du compromis a été développée.

0.5 L'Algorithme de Compromis Temps/Mémoire (Tradeoff)

Comme nous venons de le voir, l'approche fondamentale sature la mémoire vive, tandis que l'approche sans mémoire s'avère rédhibitoirement lente. Pour surmonter ce dilemme, les auteurs proposent en annexe de leur article d'exploiter la mémoire disponible (notée S) de façon stratégique pour concevoir des algorithmes de compromis (*Tradeoffs*). Ils décrivent plus précisément deux variantes distinctes, adaptées à différentes capacités matérielles :

- **Le Premier Compromis (Faible mémoire)** : Il s'agit d'une extension de l'algorithme dit « sans mémoire ». Conçu pour des environnements où l'espace S est extrêmement restreint, il trouve un petit nombre de collisions et les teste toutes de manière systématique. Cette phase de vérification exhaustive alourdit considérablement la complexité temporelle (ajoutant un terme en $\tilde{\mathcal{O}}(p^{-3})$), le rendant sous-optimal sur des machines classiques.

- **Le Second Compromis (Mémoire modérée)** : Il s'agit d'une extension de l'algorithme fondamental. Conçu pour des valeurs de S plus larges, il élimine la coûteuse phase de test systématique en cherchant directement des collisions multiples pour filtrer le bruit en amont.

Dans le cadre de ce projet, les ordinateurs modernes standards disposant généralement d'une quantité de mémoire vive confortable (de 8 à 16 Go de RAM), nous ne sommes pas dans un scénario de mémoire critique absolue. Nous nous sommes donc naturellement penchés sur le **Second Algorithme de Compromis** (*Tradeoff algorithm 2*). Cette approche permet de conserver la vitesse d'exécution optimale en $\tilde{O}(2^{n/2}p^{-1})$ tout en plafonnant intelligemment l'utilisation de la mémoire à un espace fixe dimensionné à $\tilde{O}(p^{-2})$.

0.5.1 Principe Théorique : Échantillonnage par lots (*Batching*)

Au lieu de générer et de stocker d'un seul coup un immense échantillon, l'algorithme procède par lots (*batches*). Il alloue un espace mémoire fixe S , qui correspond directement au nombre de collisions que l'on s'autorise à conserver simultanément. Pour générer ces lots, l'algorithme exécute une boucle interne s'appuyant sur la recherche de collisions parallèles (*Parallel Collision Search* ou PCS). Cette technique permet de trouver des collisions dans une fonction pseudo-aléatoire sans avoir à mémoriser toutes ses évaluations, réduisant ainsi drastiquement l'empreinte spatiale.

Pour un substitut γ donné (appelé « saveur » ou *flavor*), l'algorithme extrait donc un lot contenant exactement S collisions de la fonction g_γ . Grâce aux propriétés de la méthode PCS, l'extraction de ces S collisions s'effectue en un temps proportionnel à $2^{n/2} \cdot S^{1/2}$.

La question centrale devient alors de déterminer combien de lots successifs doivent être générés pour détecter les différentielles à haute probabilité ciblées. Pour y répondre, il faut d'abord analyser les probabilités au sein d'un seul lot et évaluer ses chances de succès.

Tout d'abord, nous savons qu'une vraie différentielle produit un « quatuor correct » (une collision utile) avec une probabilité de p^2 . Dans un lot contenant S collisions quelconques, l'espérance de trouver une collision liée à l'une de ces différentielles réelles est donc de $S \cdot p^2$. Cependant, pour éviter les faux positifs, l'algorithme ne se contente pas d'une seule bonne collision. Il exige d'en trouver deux au sein du même lot, qui suggèrent exactement la même différence d'entrée-sortie (α, β) . En probabilités, lorsque l'espérance d'un événement rare est $\lambda = S \cdot p^2$, la probabilité que cet événement se produise au moins deux fois est proportionnelle à λ^2 . La probabilité de succès pour un lot entier est donc d'environ :

$$P(\text{succès d'un lot}) \approx (S \cdot p^2)^2 = S^2 \cdot p^4$$

Puisque la probabilité d'obtenir ce résultat avec un seul lot est de $S^2 \cdot p^4$, il faudra, en moyenne, répéter le tirage un nombre de fois équivalent à l'inverse de cette probabilité pour obtenir un succès. Le nombre de lots nécessaires est donc de $S^{-2} \cdot p^{-4}$.

La complexité temporelle globale s'obtient logiquement en multipliant le temps passé à générer un seul lot par le nombre de lots nécessaires :

$$\text{Temps total} = \underbrace{\tilde{O}(2^{n/2}S^{1/2})}_{\text{Temps pour 1 lot}} \times \underbrace{S^{-2}p^{-4}}_{\text{Nombre de lots}} = \tilde{O}(2^{n/2}p^{-4}S^{-3/2})$$

L'intérêt de cette formule apparaît lorsque l'on fixe la limite de mémoire allouée à $S = \tilde{O}(p^{-2})$. En injectant cette valeur dans l'équation de la complexité temporelle, les exposants s'annulent :

$$\text{Temps total} = \tilde{O}\left(2^{n/2}p^{-4}(p^{-2})^{-3/2}\right) = \tilde{O}\left(2^{n/2}p^{-4}p^3\right) = \tilde{O}(2^{n/2}p^{-1})$$

En fragmentant ainsi la recherche, l'algorithme retrouve exactement la même complexité temporelle que l'approche fondamentale, mais en ayant réduit la complexité spatiale d'un facteur important, passant de $2^{n/2}p^{-1}$ à seulement p^{-2} .

0.5.2 De la théorie à la pratique : Notre implémentation C++

Traduire cette théorie mathématique en un programme concret nécessite quelques adaptations. Notre implémentation, librement disponible sur notre dépôt GitHub public au sein du fichier `src/analysis/DifferentialSearch.cpp` via la fonction `runMemoryEfficientAlgorithm`, gère cette contrainte de manière dynamique tout en maximisant les performances du processeur.

Le dimensionnement des lots (K^2 batches) : Le programme calcule d'abord le M_{ideal} de l'algorithme fondamental. Si cette valeur excède une limite de RAM stricte M_{MAX} fixée par le système (par exemple 100 millions d'entrées, soit environ 2,3 Go), l'algorithme bascule en mode *tradeoff*. La taille d'un lot est plafonnée à M_{MAX} , représentant un facteur de réduction $K = M_{ideal}/M_{MAX}$. Puisque le nombre de paires formées (et donc de collisions potentielles) évolue de manière quadratique selon le paradoxe des anniversaires, diviser la taille du tableau par K fait chuter le nombre de collisions générées par lot d'un facteur K^2 . Pour compenser cette perte et conserver le même taux de détection global, le programme exécute exactement K^2 lots avec des substituts γ aléatoires différents.

L'élimination de la table de hachage interne : Pour accélérer le traitement d'un lot et économiser la surcharge mémoire liée aux métadonnées des tables de hachage dynamiques, le programme stocke temporairement les évaluations dans un simple tableau plat et contigu. Une fois le lot entièrement généré, ce tableau est trié en place selon la valeur de la fonction $g_\gamma(x)$. Par conséquent, toutes les collisions se retrouvent mécaniquement adjacentes en mémoire. Il suffit alors d'effectuer un unique parcours linéaire du tableau pour les identifier et extraire les candidats. Cette approche s'avère non seulement très économe en espace, mais également extrêmement rapide car elle maximise l'utilisation de la mémoire cache du processeur.

Le Filtre de Fréquence global : Le dernier défi consiste à compter les apparitions d'un candidat (α, β) à travers les différents lots, sans recréer le goulot d'étranglement lié à une structure de hachage globale. Pour résoudre ce problème, nous avons opté pour un **filtre de fréquence** (*Frequency Filter*) pré-alloué de taille fixe (par exemple 1 Go). Lorsqu'une collision suggère une clé (α, β) , celle-ci est hachée et modulo-indexée dans ce grand tableau d'octets. Le compteur de la case correspondante est incrémenté. Ce n'est que si cette case franchit le seuil d'acceptation de l'algorithme fondamental ($n/4$) que la véritable clé (α, β) est allouée dans une liste restreinte. Cette liste finale, de très petite taille, est ensuite soumise à la phase de vérification classique.

0.5.3 Ouverture : La dépendance aux hypothèses

Grâce à cet algorithme de compromis, la détection de différentielles à haute probabilité devient réalisable sur du matériel standard, franchissant définitivement la barrière de la recherche exhaustive en 2^n .

Cependant, il est crucial de noter qu'aussi bien l'algorithme fondamental que ses variantes (sans mémoire ou de compromis) reposent sur une hypothèse fondatrice : le comportement pseudo-aléatoire de la fonction substitut g_γ . En effet, l'évaluation des probabilités, le nombre de quatuors attendus et la répartition des collisions supposent que la primitive analysée ne présente pas de structures internes venant fausser ces statistiques.

Si le chiffrement étudié possède des propriétés algébriques particulières qui empêchent g_γ de se comporter comme une fonction aléatoire, les garanties mathématiques de détection s'effondrent. Cette dépendance aux hypothèses justifie la nécessité de concevoir un ultime filet de sécurité : un algorithme « pire cas » (*Worst-Case Algorithm*). Bien que plus lourd, ce dernier doit être capable de retrouver ces propriétés statistiques sans formuler la moindre hypothèse sur le comportement structurel de la primitive étudiée.

0.6 L'Algorithme « Pire Cas » (*Worst-Case*) : S'affranchir des heuristiques

Comme évoqué précédemment, les algorithmes fondamental et de compromis reposent sur un pari fort : l'hypothèse que les quatuors corrects se distribuent de manière aléatoire au sein de l'espace des messages. Cependant, si un concepteur malveillant introduit une porte dérobée (*trapdoor*) dans le chiffrement, par exemple, en forçant les paires correctes à se concentrer dans un sous-espace linéaire restreint, l'algorithme fondamental échouera presque systématiquement, car il n'aura testé qu'un ou très peu de substituts γ .

Pour contrer cette menace et garantir une détection infailible même face à une primitive conçue de manière adverse, les auteurs de l'article introduisent un algorithme dit « pire cas ». Son principe est de ne plus s'en remettre à la chance sur le choix du substitut, mais de forcer statistiquement la découverte en multipliant les angles d'attaque de manière indépendante.

0.6.1 Paramétrage et seuils de validation

Au lieu d'utiliser un échantillon géant avec un seul substitut γ (ou de chercher jusqu'à trouver un succès comme dans le tradeoff), l'algorithme *Worst-Case* va tirer un nombre prédéfini et massif de substituts γ différents, et générer un sous-échantillon pour chacun d'eux. La mécanique repose sur trois paramètres mathématiques interconnectés, rigoureusement calibrés pour isoler le signal du bruit sans formuler d'hypothèses sur la distribution.

1. La taille du sous-échantillon M : La première différence majeure avec l'algorithme fondamental réside dans le dimensionnement de M . L'algorithme fondamental cherchait à accumuler suffisamment de collisions globales en une seule fois, exigeant un M colossal. Ici, l'objectif d'un seul lot est beaucoup plus modeste : pour un substitut γ donné, le lot doit être juste assez grand pour garantir la présence d'**au moins un** quatuor correct (ce qui constituera un « vote » pour ce γ).

Pour garantir une forte probabilité de succès (fixée par les auteurs à 80 %, soit $4/5$), ces derniers s'appuient sur la méthode probabiliste du second moment dans la démonstration de leur Lemme 1. Cette méthode indique que pour atteindre le seuil de $4/5$, l'espérance mathématique du nombre de quatuors corrects doit être d'au moins 8. Sachant que cette espérance se calcule par $E = \frac{M^2}{2} \cdot 2^{-n} \cdot p$, on peut en déduire la taille optimale du lot :

$$\frac{M^2}{2} \cdot 2^{-n} \cdot p = 8 \implies M^2 = 16 \cdot 2^n \cdot p^{-1} \implies M = 4 \cdot 2^{n/2} \cdot p^{-1/2}$$

C'est pourquoi M évolue ici en $p^{-1/2}$ et non plus en p^{-1} , rendant chaque lot beaucoup plus léger à stocker en mémoire.

2. Le seuil d'acceptation global ($0,28 \cdot S \cdot p$) : À la fin du traitement d'un lot, si une différence (α, β) a été repérée, le substitut γ donne « un vote » pour ce candidat (un seul vote maximum par γ , pour éviter qu'un sous-espace dense ne fausse le comptage). Le programme doit ensuite décider combien de votes sont nécessaires sur l'ensemble des itérations pour valider une différentielle.

Ce seuil est obtenu en calculant l'espérance des votes d'un vrai signal par rapport à un bruit statistique :

- **Pour une vraie différentielle (probabilité $\geq p$) :** Comme établi par la taille de M , le lot a déjà une probabilité de $4/5$ de contenir une bonne paire initiale. La question est de savoir si la paire translatée par le substitut aléatoire γ sera elle aussi correcte. Dans un scénario idéal, cette probabilité conditionnelle serait de p . Toutefois, pour garantir que l'algorithme fonctionne même dans le pire des cas (où un adversaire aurait biaisé la distribution), les auteurs démontrent mathématiquement que cette probabilité ne peut jamais descendre sous la borne stricte de $p/2$. La probabilité minimale qu'un lot génère un vote est donc le

produit de ces deux étapes : $(4/5) \times (p/2) = 2p/5 = 0,40 \cdot p$. Sur S itérations, on s'attend à récolter en moyenne $0,40 \cdot S \cdot p$ votes.

- **Pour une fausse différentielle (probabilité $\leq p/10$)** : Ici, aucune paire n'est garantie par la taille du lot. La probabilité qu'une fausse piste forme un quatuor par pur hasard nécessite de valider les deux paires consécutivement, ce qui force à élever sa probabilité au carré. L'évaluation de l'espérance (qui dépend de M^2 , introduisant ainsi un facteur 16 puisque M commence par 4) donne un plafond de $16 \cdot p \cdot (1/10)^2 = 0,16 \cdot p$. L'espérance maximale des votes pour un bruit est donc de $0,16 \cdot S \cdot p$.

Pour séparer le signal du bruit, le seuil d'acceptation est fixé exactement à la moyenne arithmétique entre ces deux espérances (0,16 et 0,40) :

$$\text{Seuil} = \frac{0,40 + 0,16}{2} \cdot S \cdot p = 0,28 \cdot S \cdot p$$

3. Le nombre d'itérations S : Fixer une moyenne parfaite ne suffit pas : il faut répéter l'expérience un nombre de fois S suffisamment grand pour se prémunir des fluctuations statistiques. Sachant qu'il existe 2^{2n} combinaisons possibles de fausses différentielles (α, β) , la probabilité qu'une seule d'entre elles franchisse le seuil par erreur doit être infime (strictement inférieure à 2^{-2n}).

Dans le Lemme 2 de l'article, les auteurs utilisent la **borne de Chernoff** pour évaluer cette probabilité d'erreur, qui s'exprime sous la forme exponentielle $e^{-\frac{S \cdot p}{50}}$. Afin de sécuriser le résultat face aux 2^{2n} fausses pistes, ils choisissent d'imposer un taux d'erreur encore plus strict, fixé à e^{-4n} (soit environ $2^{-5,7n}$). Pour trouver le nombre d'itérations S nécessaire, il suffit de résoudre l'égalité entre ces deux exposants :

$$\frac{S \cdot p}{50} = 4n \implies S \cdot p = 200n \implies S = \frac{200n}{p}$$

Le paramètre constant 200 découle directement de cette contrainte de sécurité. En remplaçant S dans notre équation de seuil, on obtient finalement la condition de validation implémentée dans notre code : un candidat est validé s'il obtient plus de $0,28 \cdot \left(\frac{200n}{p}\right) \cdot p = 56n$ votes.

0.6.2 Complexités théoriques et Implémentation

Cette robustesse absolue a un coût. La complexité temporelle globale de l'algorithme s'obtient en multipliant le nombre d'itérations S par la taille du lot M à évaluer à chaque fois :

$$\text{Temps} = \tilde{O}(S \cdot M) = \tilde{O}\left(p^{-1} \times 2^{n/2} p^{-1/2}\right) = \tilde{O}(2^{n/2} p^{-3/2})$$

Par rapport à l'algorithme fondamental $\tilde{O}(2^{n/2} p^{-1})$, cette approche pire cas subit un ralentissement d'un facteur $\tilde{O}(p^{-1/2})$. En revanche, sa complexité spatiale est limitée au stockage de la table de hachage d'un seul lot M à la fois, soit $\tilde{O}(2^{n/2} p^{-1/2})$, ce qui représente un excellent compromis.

Notre implémentation C++ de cette méthode, encapsulée dans la fonction `runWorstCaseAlgorithm`, exploite pleinement la nature indépendante des itérations S . La boucle principale gérant les différents substituts γ a été parallélisée. Chaque thread alloue sa propre table de hachage temporaire pour analyser un sous-échantillon M . Il ne vient mettre à jour le grand compteur global des votes qu'à la fin de son évaluation. Une fois les S itérations terminées, le programme applique le filtre final des $56n$ votes et exporte les différentielles certifiées.

0.6.3 Limites pratiques : Le cumul des contraintes

Bien que cette variante offre des garanties théoriques absolues de sécurité, elle cumule en pratique les défauts majeurs des deux algorithmes précédents : une forte consommation de

RAM (héritée de l'algorithme fondamental) et une grande lenteur (héritée de l'approche sans mémoire).

En effet, le ralentissement théorique d'un facteur $p^{-1/2}$ et la nécessité de stocker M messages en mémoire simultanément rendent l'exécution extrêmement lourde. Pour s'en rendre compte, reprenons notre exemple numérique sur le chiffrement Speck 32/64 ($n = 32$) avec une probabilité $p = 2^{-13}$:

- **L'obstacle temporel** : Le facteur de ralentissement $p^{-1/2}$ vaut ici $\sqrt{2^{13}} \approx 90,5$. Cela signifie qu'une recherche qui prenait **1 minute** avec l'algorithme fondamental prendra ici **plus d'une heure et demie**.
- **L'obstacle spatial (RAM)** : Pour ces paramètres, la taille d'un sous-échantillon M atteint environ 23,7 millions de messages. En pratique, une table de hachage pèse environ 32 octets par entrée en raison des métadonnées requises par la structure de données sous-jacente (gestion des collisions, pointeurs internes). Chaque fil d'exécution consomme donc près de 750 Mo. Sur un processeur standard exécutant 8 fils en parallèle, l'algorithme accapare instantanément **6 Go de mémoire vive** pour analyser un simple chiffrement « jouet » de 32 bits.

Une piste d'amélioration théorique (non implémentée dans nos travaux) consisterait à appliquer la méthode de recherche de collisions parallèles (PCS) au sein même de cet algorithme « pire cas », à l'instar de ce qui a été fait pour le compromis temps/mémoire. Cela permettrait d'éliminer le stockage direct des M messages dans des tables de hachage et réduirait drastiquement l'empreinte spatiale. Néanmoins, bien que le problème de la RAM puisse être atténué par cette astuce, la barrière temporelle imposée par la complexité en $\tilde{O}(2^{n/2}p^{-3/2})$ resterait un obstacle infranchissable pour analyser des chiffrements avec des tailles de blocs standards ($n \geq 64$).

0.7 Choix d'implémentation : Optimisation pour les blocs de 32 bits

Tout au long du développement de nos algorithmes, nous avons dû faire des choix architecturaux pour maximiser la vitesse d'exécution et minimiser l'empreinte mémoire. L'une de nos optimisations principales concerne le stockage et l'indexation des candidats différentiels (α, β) .

Dans les algorithmes de recherche de collisions, le programme doit constamment stocker, chercher et compter les occurrences des couples d'entrée-sortie. Représenter ces informations sous forme de paires d'objets distincts exigerait des fonctions de hachage complexes, ce qui introduirait une surcharge de calcul non négligeable à l'échelle de millions d'accès mémoire par seconde.

Pour contourner ce problème et accélérer les traitements, nous avons opté pour une technique de concaténation binaire. Le principe consiste à fusionner la différence d'entrée α et la différence de sortie β au sein d'un seul et unique entier non signé de 64 bits. Mathématiquement, cette fusion s'opère en décalant les bits de α vers la gauche pour les placer sur la moitié forte de l'entier, puis en appliquant une disjonction logique (OU binaire) avec β sur la moitié faible :

$$\text{Clé globale (64 bits)} = (\alpha \ll 32) \vee \beta$$

En utilisant cette clé numérique unique, le comptage des candidats devient extrêmement rapide. Un tel entier est en effet nativement et parfaitement pris en charge par les registres du processeur et par les algorithmes de hachage internes, garantissant des performances d'indexation optimales.

Cependant, cette optimisation matérielle induit une limite structurelle stricte : puisque l'entier global fait 64 bits, il ne peut contenir que deux valeurs de 32 bits maximum. Notre base de code est donc intrinsèquement optimisée pour l'analyse de chiffrements ayant une taille de bloc $n \leq 32$ bits (à l'image de Speck 32/64).

Pour adapter notre programme à des chiffrements aux standards actuels ($n = 64$ ou $n = 128$ bits), il faudrait abandonner cette optimisation au profit d'entiers beaucoup plus larges (comme des types de 128 ou 256 bits) ou revenir à l'utilisation de paires structurelles. Dans les deux

cas, cela impacterait mécaniquement l’empreinte mémoire et la vitesse d’exécution brute de la recherche.

0.8 Validation expérimentale et résolution matérielle

Pour valider notre implémentation, nous avons entrepris de reproduire les résultats expérimentaux de l’article de référence (Section 2.5). L’objectif était de redécouvrir de manière automatisée la meilleure caractéristique différentielle sur 5 tours de Speck 32/64. L’article identifie le couple suivant, dont la probabilité théorique est de l’ordre de $p \approx 2^{-9}$:

$$(0x0211, 0x0a04) \rightarrow (0x8000, 0x840a)$$

Lors de nos premières exécutions avec l’algorithme fondamental, nous nous sommes rapidement heurtés à des limites matérielles. Pour atteindre une probabilité de 2^{-9} , la taille de l’échantillon d’évaluation M s’élevait à environ 189 millions de messages. Le maintien d’une table de hachage pour un tel volume a entraîné une surcharge massive de la mémoire vive, due à la taille des données nécessaires à la gestion des collisions internes de la structure. Cela a provoqué la saturation de la mémoire et l’arrêt du programme.

Dans un premier temps, nous avons tenté de contourner ce problème d’allocation en remplaçant la table de hachage par un simple tableau plat et contigu. L’idée était d’évaluer le lot, de trier le tableau, puis d’en extraire les collisions par un parcours linéaire. Si cette approche s’est révélée très économe en mémoire et parfaitement optimisée pour le cache processeur, elle modifiait la complexité temporelle et la logique originelle de l’algorithme.

C’est une relecture de l’Annexe A du papier qui nous a fourni la solution algorithmique adéquate : l’algorithme de compromis Temps/Mémoire. En implémentant cette variante, nous avons pu traiter les évaluations par lots limités à 100 millions de textes clairs. L’empreinte mémoire s’est ainsi stabilisée autour de 2,2 Go par lot, permettant au programme de s’exécuter jusqu’à son terme de manière stable.

Afin de lisser la forte dépendance à la clé inhérente au chiffrement, l’algorithme a été exécuté sur un échantillon de 1000 clés générées aléatoirement. Cette campagne complète a nécessité un temps de calcul de 109 538 secondes (environ 30 heures).

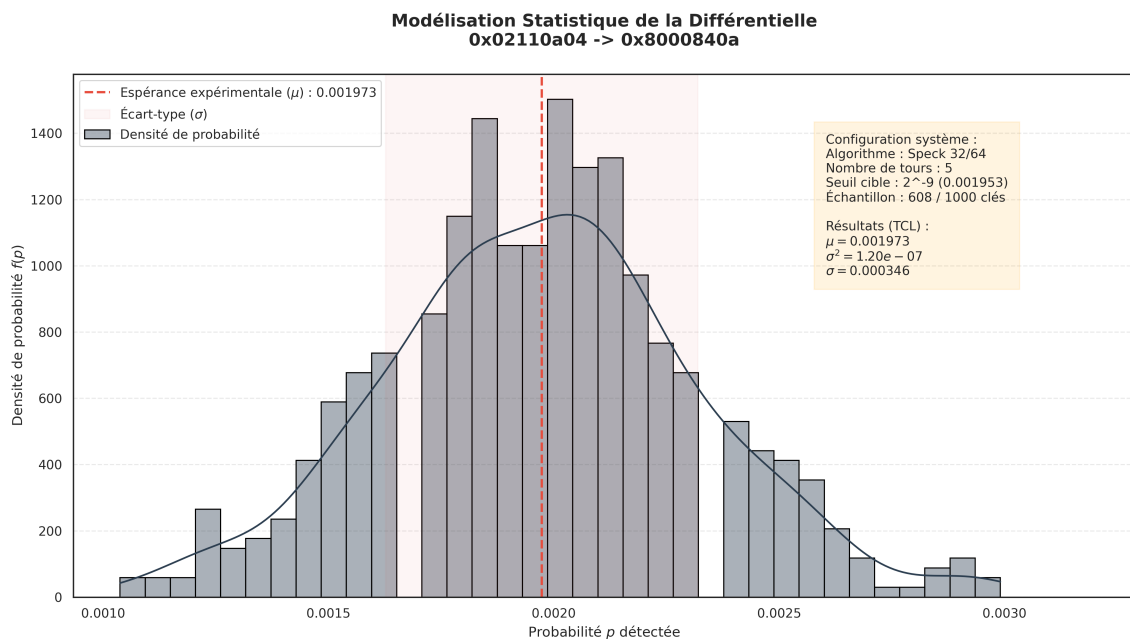


Figure 1 – Distribution de la densité de la probabilité différentielle sur 1000 clés (Speck 32/64, 5 tours). La modélisation confirme la convergence vers une loi normale centrée sur $p \approx 2^{-9}$.

Les données récoltées ont été modélisées sous forme de distribution de densité (voir Figure 1). Contrairement à un simple histogramme de comptage brut, la normalisation en densité contraint l'aire totale sous la courbe à une valeur de 1. Ce choix méthodologique permet de comparer rigoureusement la forme de notre distribution indépendamment de la taille de l'échantillon. Nous y avons superposé une estimation par noyau afin de modéliser visuellement le comportement continu théorique du chiffrement.

L'analyse de ce graphique confirme empiriquement l'application du Théorème Central Limite (TCL). En considérant la probabilité de la caractéristique pour une clé aléatoire comme une variable aléatoire, notre échantillon massif démontre une convergence claire vers une loi normale $\mathcal{N}(\mu, \sigma^2)$. L'espérance mesurée se fixe à $\mu \approx 0,00197$, ce qui valide avec une excellente précision la probabilité théorique de 2^{-9} (soit 0,001953) attendue par l'état de l'art.

Cependant, une analyse plus fine de la variance révèle une particularité structurelle importante. Même si l'écart-type mesuré reste très restreint ($\sigma \approx 3,46 \cdot 10^{-4}$), l'échantillon indique que seules 608 des 1000 clés testées ont produit un signal atteignant notre seuil de détection. Les 39,2 % de clés restantes n'ont pas disparu : elles constituent la queue de distribution gauche (les probabilités inférieures à 2^{-9}), coupée de notre graphique par le filtre d'acceptation de l'algorithme. Ce phénomène met en évidence l'existence d'une proportion significative de « clés fortes ». Pour ces dernières, la propagation de cette différentielle spécifique est atténuée, illustrant la dépendance à la clé de l'algorithme Speck sur un nombre restreint de tours.